

Методические указания к выполнению лабораторной работы № 2

Векторизация алгоритмов обработки изображений и линейной алгебры

Направление подготовки: 09.04.02 «Информационные системы и технологии»

Содержание

1	Тема лабораторной работы	2
2	Цель и задачи лабораторной работы	2
2.1	Цель работы	2
2.2	Задачи работы	2
3	Программное обеспечение	3
4	Введение	3
4.1	Актуальность работы	3
4.2	Обзор научных работ по теме	4
4.3	Постановка задачи	5
5	Теоретические сведения	5
5.1	Архитектура SIMD и векторные регистры	5
5.2	Модель Roofline: Compute-bound vs Memory-bound	5
5.3	Авто-векторизация компилятора	6
5.4	Векторные интринсики	6
5.5	Пространственная локальность и Loop Interchange	7
5.6	Математическая модель умножения матриц	7
5.7	Математическая модель двумерной свёртки	8
5.8	Описание функций NumPy для векторизации	8
5.9	Описание функций Numba для JIT-компиляции	8
5.10	Методы профилирования	9
6	Программа выполнения лабораторной работы	9
6.1	Задача 1. Умножение матриц	9
6.1.1	Описание задачи из предметной области	9
6.1.2	План решения задачи	10
6.1.3	Подробное пошаговое описание решения	10
6.1.4	Программный код на Python	10
6.1.5	Программный код на C++	11
6.1.6	Анализ решения и формулирование выводов	11
6.2	Задача 2. Двумерная свёртка	11
6.2.1	Описание задачи из предметной области	11

6.2.2	План решения задачи	12
6.2.3	Подробное пошаговое описание решения	12
6.2.4	Программный код на Python	13
6.2.5	Программный код на C++	13
6.2.6	Анализ решения и формулирование выводов	13
6.3	Задача 3. Профилирование и оптимизация выравнивания данных	13
6.3.1	Описание задачи из предметной области	13
6.3.2	План решения задачи	13
6.3.3	Подробное пошаговое описание решения	14
6.3.4	Программный код на Python	14
6.3.5	Программный код на C++	14
6.3.6	Анализ решения и формулирование выводов	14
7	Индивидуальное задание	15
8	Тестовые задания с ответами	16
8.1	Задания с выбором одного варианта из нескольких	16
8.2	Задания с множественным выбором	16
8.3	Задания на сопоставление	17
8.4	Задания на ввод пропущенного слова	18
8.5	Задания на анализ фрагмента программного кода	18
8.6	Задания на поиск ошибок в фрагменте программного кода	19
9	Контрольные вопросы	19
A	Фрагменты программного кода	20
A.1	Фрагмент 1. Скалярное умножение матриц на Python	20
A.2	Фрагмент 2. Векторизованное умножение матриц с NumPy	20
A.3	Фрагмент 3. JIT-компиляция умножения матриц с Numba	21
A.4	Фрагмент 4. Скалярное умножение матриц на C++	21
A.5	Фрагмент 5. Умножение матриц с авто-векторизацией C++	22
A.6	Фрагмент 6. Умножение матриц с интринсиками AVX2	23
A.7	Фрагмент 7. Блочное умножение матриц на C++	23
A.8	Фрагмент 8. Скалярная свёртка на Python	24
A.9	Фрагмент 9. Свёртка с SciPy	25
A.10	Фрагмент 10. Свёртка с Numba	25
A.11	Фрагмент 11. Скалярная свёртка на C++	26
A.12	Фрагмент 12. Свёртка с авто-векторизацией C++	26
A.13	Фрагмент 13. Свёртка с интринсиками AVX2	27
A.14	Фрагмент 14. Свёртка с разделяемым ядром на C++	29
A.15	Фрагмент 15. Блочная обработка на Python	30
A.16	Фрагмент 16. Блочная обработка с выравниванием на C++	30
A.17	Фрагмент 17. Блочная обработка без выравнивания на C++	31
A.18	Фрагмент 18. Фрагмент для анализа (задание 21)	32
A.19	Фрагмент 19. Фрагмент для анализа (задание 22)	33
A.20	Фрагмент 20. Фрагмент для анализа (задание 23)	33
A.21	Фрагмент 21. Фрагмент для анализа (задание 24)	33
A.22	Фрагмент 22. Фрагмент для анализа (задание 25)	34
A.23	Фрагмент 23. Фрагмент с ошибкой (задание 26)	34
A.24	Фрагмент 24. Фрагмент с ошибкой (задание 27)	34
A.25	Фрагмент 25. Фрагмент с ошибкой (задание 28)	34

A.26 Фрагмент 26. Фрагмент с ошибкой (задание 29)	35
A.27 Фрагмент 27. Фрагмент с ошибкой (задание 30)	35

1. Тема лабораторной работы

Векторизация алгоритмов обработки изображений и линейной алгебры: реализация алгоритмов умножения матриц и двумерной свёртки с использованием векторных внутренних функций (интринсиков), анализ производительности, профилирование и оптимизация выравнивания данных.

2. Цель и задачи лабораторной работы

2.1. Цель работы

Целью лабораторной работы является освоение практических навыков применения векторизации для ускорения алгоритмов обработки спутниковых изображений и линейной алгебры на языках Python и C++, а также приобретение компетенций в области анализа производительности, профилирования и оптимизации программного обеспечения для высокопроизводительных вычислений.

2.2. Задачи работы

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить теоретические основы SIMD-векторизации, включая архитектурные особенности процессоров x86-64 и ARM, поддерживающих наборы инструкций SSE, AVX2 и AVX-512.
2. Освоить математические модели умножения матриц и двумерной свёртки применительно к задачам обработки спутниковых изображений.
3. Реализовать базовые (скалярные) версии алгоритмов умножения матриц и двумерной свёртки на языках Python и C++.
4. Реализовать векторизованные версии алгоритмов с использованием авто-векторизации компилятора и явных векторных интринсиков.
5. Провести сравнительный анализ производительности скалярных и векторизованных реализаций.
6. Освоить методы профилирования с использованием системных средств анализа производительности (perf, VTune, gprof).
7. Измерить утилизацию векторных регистров и количество промахов кэша.
8. Исследовать влияние выравнивания данных в памяти и порядка циклов (Loop Interchange) на производительность векторизованных алгоритмов.
9. Сформулировать рекомендации по оптимизации алгоритмов обработки изображений для современных процессорных архитектур.

3. Программное обеспечение

Для выполнения лабораторной работы требуется следующее программное обеспечение:

№	Наименование	Назначение	Версия
1	Python	Интерпретируемый язык программирования	3.10+
2	NumPy	Библиотека для численных вычислений	1.24+
3	SciPy	Библиотека для научных вычислений	1.11+
4	Numba	JIT-компилятор для Python	0.58+
5	Cython	Компилятор Python-подобного кода в C	3.0+
6	GCC	Компилятор C++	12.0+
7	Clang	Альтернативный компилятор C++	16.0+
8	CMake	Система сборки	3.25+
9	perf	Системный профилировщик (Linux)	6.0+
10	Intel VTune	Профилировщик производительности	2024+
11	Valgrind / Cachegrind	Анализ промахов кэша	3.20+
12	GDAL	Библиотека для геопространственных данных	3.7+
13	OpenCV	Библиотека компьютерного зрения	4.9+
14	Matplotlib	Библиотека визуализации	3.8+

Таблица 1: Программное обеспечение

Требования к аппаратному обеспечению:

- Процессор с поддержкой AVX2 или AVX-512 (Intel Core i7/i9 10-го поколения и выше, AMD Ryzen 5000 и выше)
- Не менее 16 ГБ оперативной памяти
- Не менее 50 ГБ свободного дискового пространства
- Операционная система: Linux (Ubuntu 22.04 LTS или выше)

4. Введение

4.1. Актуальность работы

Современные системы дистанционного зондирования Земли (ДЗЗ) генерируют гигабайты данных ежедневно. Спутниковые изображения высокого разрешения, получаемые с аппаратов серии Sentinel, Landsat, WorldView и российских «Ресурс-П», «Канопус-В», требуют оперативной обработки для решения задач мониторинга сельскохозяйственных

угодий, лесных массивов, водных ресурсов и городской инфраструктуры. В условиях Азово-Черноморского региона особую значимость приобретают задачи мониторинга сельскохозяйственных культур, оценки состояния виноградников и садов, контроля эрозионных процессов и анализа прибрежных зон.

Вычислительная сложность алгоритмов обработки спутниковых изображений растёт экспоненциально с увеличением пространственного разрешения. Если для изображения Sentinel-2 размером 10980×10980 пикселей операция двумерной свёртки с ядром 5×5 требует порядка 3×10^9 операций умножения-сложения (FMA), то для гиперспектральных изображений с сотнями спектральных каналов эта величина возрастает на два-три порядка. Традиционные скалярные реализации оказываются неспособными обеспечить обработку таких объёмов данных в режиме, близком к реальному времени.

Технология SIMD (Single Instruction, Multiple Data) позволяет обрабатывать несколько элементов данных одной инструкцией, обеспечивая ускорение в 4–16 раз для 256-битных и 512-битных регистров. Современные процессоры x86-64 поддерживают наборы инструкций AVX2 (256 бит) и AVX-512 (512 бит), позволяющие одновременно обрабатывать 8 и 16 чисел с плавающей точкой одинарной точности соответственно. Использование векторных интринсиков и авто-векторизации компилятора позволяет существенно сократить время обработки спутниковых изображений.

Актуальность настоящей работы обусловлена необходимостью подготовки специалистов, способных разрабатывать высокопроизводительные алгоритмы обработки спутниковых изображений с использованием современных методов векторизации и оптимизации.

4.2. Обзор научных работ по теме

Проблематика векторизации алгоритмов обработки изображений активно исследуется в ведущих научных журналах. Рассмотрим наиболее значимые публикации последних лет.

В работе «High-performance matrix multiplication on modern multi-core processors» (IEEE Transactions on Parallel and Distributed Systems, 2023) авторы исследуют методы оптимизации умножения матриц с использованием AVX-512 и многоуровневого кэширования. Показано, что комбинация блочного умножения, векторизации и предварительной выборки данных позволяет достичь производительности, близкой к теоретическому пику процессора.

В статье «SIMD-accelerated convolution for real-time image processing» (Journal of Real-Time Image Processing, 2022) рассматриваются подходы к векторизации двумерной свёртки для задач обработки изображений в реальном времени. Авторы демонстрируют, что использование разделяемых ядер и векторизации позволяет достичь ускорения в 8–12 раз по сравнению со скалярной реализацией.

В публикации «Auto-vectorization techniques for embedded image processing» (ACM Transactions on Embedded Computing Systems, 2023) анализируются возможности современных компиляторов по автоматической векторизации циклов обработки изображений. Показано, что правильное выравнивание данных и устранение зависимостей по данным позволяют компилятору генерировать эффективный векторный код без явного использования интринсиков.

В работе «Cache-aware optimization of satellite image processing algorithms» (Remote Sensing, 2024) исследуется влияние промахов кэша на производительность алгоритмов обработки спутниковых изображений. Авторы предлагают методику блочной обработки, позволяющую минимизировать количество обращений к оперативной памяти и повысить эффективность использования кэша.

В статье «Vectorization of hyperspectral image processing using AVX-512» (IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, 2024) рассматриваются

особенности векторизации гиперспектральных изображений с использованием 512-битных регистров. Показано, что для гиперспектральных данных векторизация даёт наибольший эффект благодаря высокой плотности вычислений.

В публикации «Performance analysis of Python libraries for scientific computing» (Computing in Science & Engineering, 2023) проводится сравнительный анализ производительности NumPy, Numba и Cython для задач линейной алгебры. Авторы показывают, что для вычислительно интенсивных задач использование Cython и Numba позволяет достичь производительности, сопоставимой с C++.

4.3. Постановка задачи

В рамках настоящей лабораторной работы рассматриваются две ключевые задачи обработки спутниковых изображений:

- 1. **Умножение матриц** — базовая операция линейной алгебры, используемая в спектральном анализе, методах главных компонент и нейросетевых моделях.
- 2. **Двумерная свёртка** — операция фильтрации изображений, применяемая для сглаживания, выделения границ и обнаружения объектов на спутниковых снимках.

Для каждой задачи необходимо реализовать скалярные и векторизованные версии на языках Python и C++, провести сравнительный анализ производительности и сформулировать рекомендации по оптимизации.

5. Теоретические сведения

5.1. Архитектура SIMD и векторные регистры

SIMD (Single Instruction, Multiple Data) — принцип параллельной обработки данных, при котором одна инструкция применяется к нескольким элементам данных одновременно. Современные процессоры x86-64 поддерживают следующие наборы SIMD-инструкций:

Набор	Год	Ширина регистра	Элементов float	Элементов double
SSE	1999	128 бит	4	2
AVX	2011	256 бит	8	4
AVX2	2013	256 бит	8	4
AVX-512	2016	512 бит	16	8

Таблица 2: Наборы SIMD-инструкций x86-64

Векторные регистры `ymm` (256 бит) и `zmm` (512 бит) позволяют обрабатывать 8 и 16 чисел с плавающей точкой одинарной точности соответственно. Операции над векторными регистрами выполняются на специальных функциональных устройствах (Execution Units), что обеспечивает высокую пропускную способность.

5.2. Модель Roofline: Compute-bound vs Memory-bound

При оптимизации алгоритмов важно понимать, в какой ресурс упирается производительность:

- **Compute-bound (Вычислительное ограничение):** Алгоритм выполняет много арифметических операций на каждый байт загруженных данных. *Пример: Умножение матриц.* Арифметическая интенсивность растёт с размером $O(N)$. Векторизация (FMA) даёт кратный рост скорости.
- **Memory-bound (Ограничение памяти):** Алгоритм выполняет мало операций на каждый байт. *Пример: Свёртка с малым ядром (3x3).* Интенсивность $O(1)$. Скорость упирается в пропускную способность RAM/кэша. Здесь критически важен **Cache Blocking** и **Prefetching**.

5.3. Авто-векторизация компилятора

Современные компиляторы GCC и Clang способны автоматически векторизовать циклы, если выполняются следующие условия:

1. Отсутствуют зависимости по данным между итерациями цикла.
2. Количество итераций известно на этапе компиляции или может быть вычислено.
3. Используются простые типы данных, поддерживающие векторизацию.
4. Отсутствуют вызовы функций, не поддерживающих векторизацию.

Флаги компиляции для включения авто-векторизации:

- **GCC:**
 - `-O3` — включает авто-векторизацию по умолчанию.
 - `-ftree-vectorize` — явно включает векторизацию.
 - `-march=native` — использует все инструкции, доступные на текущем процессоре.
 - `-fopt-info-vec-optimized` — выводит отчёт о векторизованных циклах.
 - `-fopt-info-vec-missed` — выводит отчёт о не векторизованных циклах.
- **Clang:**
 - `-O3 -fvectorize` — включает векторизацию.
 - `-Rpass=loop-vectorize` — отчёт об успешной векторизации.
 - `-Rpass-missed=loop-vectorize` — отчёт о причинах отказа от векторизации.

5.4. Векторные интринсики

Векторные интринсики — это встроенные функции компилятора, соответствующие SIMD-инструкциям процессора. Заголовочный файл `<immintrin.h>` предоставляет доступ ко всем наборам инструкций x86. Основные типы данных:

- `__m128` — 128-битный вектор ($4 \times \text{float}$)
- `__m256` — 256-битный вектор ($8 \times \text{float}$)
- `__m512` — 512-битный вектор ($16 \times \text{float}$)

Для устранения псевдонимов указателей (pointer aliasing) и помощи компилятору в оптимизации рекомендуется использовать ключевое слово `__restrict` (или `__restrict__` в GCC) при передаче указателей в функции.

Основные операции:

- `_mm256_load_ps(ptr)` — загрузка выровненных данных (требует выравнивания по 32 байтам).
- `_mm256_loadu_ps(ptr)` — загрузка невыровненных данных (безопасно).
- `_mm256_store_ps(ptr, vec)` — сохранение выровненных данных.
- `_mm256_add_ps(a, b)` — векторное сложение.
- `_mm256_mul_ps(a, b)` — векторное умножение.
- `_mm256_fmadd_ps(a, b, c)` — FMA: $a \times b + c$.
- `_mm256_setzero_ps()` — обнуление вектора.
- `_mm256_set1_ps(val)` — заполнение всех элементов вектора значением `val` (broadcasting).

5.5. Пространственная локальность и Loop Interchange

В языках C/C++ и библиотеке NumPy многомерные массивы хранятся **по строкам (row-major)**. Это означает, что элементы `A[i][j]` и `A[i][j+1]` лежат в памяти рядом. **Правило оптимизации:** Внутренний цикл должен итерироваться по последнему индексу (по строке).

Плохо (промахи кэша, шаг по памяти N):

```

1 for (int i = 0; i < M; ++i)
2     for (int j = 0; j < N; ++j)
3         for (int k = 0; k < K; ++k)
4             C[i][j] += A[i][k] * B[k][j];

```

Хорошо (последовательный доступ, шаг 1):

```

1 for (int i = 0; i < M; ++i)
2     for (int k = 0; k < K; ++k)
3         for (int j = 0; j < N; ++j)
4             C[i][j] += A[i][k] * B[k][j];

```

5.6. Математическая модель умножения матриц

Умножение матриц определяется следующим образом. Пусть даны матрицы $A \in \mathbb{R}^{M \times K}$ и $B \in \mathbb{R}^{K \times N}$. Тогда матрица $C = A \cdot B \in \mathbb{R}^{M \times N}$ определяется поэлементно:

$$C_{ij} = \sum_{k=1}^K A_{ik} \cdot B_{kj}, \quad i = 1, \dots, M, \quad j = 1, \dots, N. \quad (1)$$

Вычислительная сложность прямого алгоритма составляет $O(MNK)$ операций умножения-сложения. Для спутниковых изображений с матрицами размером 10000×10000 это составляет 10^{12} операций, что требует применения векторизации и блочных алгоритмов.

Блочный алгоритм умножения матриц разбивает исходные матрицы на блоки размером $B \times B$, помещающиеся в кэш, что позволяет минимизировать количество обращений к оперативной памяти:

$$C_{I,J} = \sum_{K=1}^{\lceil K_{total}/B \rceil} A_{I,K} \cdot B_{K,J}, \quad (2)$$

где I, J, K — индексы блоков.

5.7. Математическая модель двумерной свёртки

Двумерная свёртка изображения I с ядром K определяется как:

$$(I * K)(x, y) = \sum_{i=-a}^a \sum_{j=-b}^b I(x-i, y-j) \cdot K(i, j), \quad (3)$$

где $a = \lfloor w/2 \rfloor$, $b = \lfloor h/2 \rfloor$, $w \times h$ — размер ядра.

Примечание: В компьютерном зрении (OpenCV, нейросети) фактически используется **кросс-корреляция** (без переворота ядра на 180°). Для симметричных ядер (Гаусс, Собель, Лаплас) $K(i, j) = K(-i, -j)$, поэтому результаты математической свёртки и кросс-корреляции идентичны. В данной работе мы реализуем кросс-корреляцию, так как она требует меньше вычислений.

Разделяемые ядра. Если ядро K представимо в виде произведения двух одномерных ядер $K(i, j) = K_x(i) \cdot K_y(j)$, то двумерная свёртка может быть вычислена за два прохода:

$$(I * K)(x, y) = \sum_{i=-a}^a \left(\sum_{j=-b}^b I(x-i, y-j) \cdot K_y(j) \right) \cdot K_x(i). \quad (4)$$

Это позволяет сократить вычислительную сложность с $O(N^2wh)$ до $O(N^2(w+h))$.

Оператор Собеля. Для выделения границ на спутниковых изображениях широко используется оператор Собеля:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}. \quad (5)$$

Градиент изображения вычисляется как:

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}. \quad (6)$$

5.8. Описание функций NumPy для векторизации

numpy.dot(a, b) — умножение матриц. Использует оптимизированные библиотеки BLAS (OpenBLAS, MKL). Внутренняя реализация задействует SIMD-инструкции.

numpy.convolve(a, v, mode) — одномерная свёртка. Для двумерной свёртки используется `scipy.signal.convolve2d`.

numpy.einsum(equation, *operands) — вычисление выражений Эйнштейна. Позволяет компактно записывать сложные операции тензорной алгебры.

5.9. Описание функций Numba для JIT-компиляции

numba.njit(signature) — декоратор для JIT-компиляции функций. Позволяет достичь производительности, близкой к C, при сохранении синтаксиса Python.

numba.vectorize — декоратор для создания универсальных функций (ufunc), работающих поэлементно.

numba.prange — параллельный диапазон для автоматического распараллеливания циклов.

5.10. Методы профилирования

perf stat — сбор статистики о производительности программы:

- **cache-references** — количество обращений к кэшу.
- **cache-misses** — количество промахов кэша.
- **instructions** — количество выполненных инструкций.
- **cycles** — количество тактов процессора.

perf record — запись профиля выполнения программы для последующего анализа.

perf annotate — аннотирование исходного кода с указанием времени выполнения отдельных инструкций.

Cachegrind — инструмент Valgrind для детального анализа промахов кэша по уровням L1, L2, L3.

6. Программа выполнения лабораторной работы

6.1. Задача 1. Умножение матриц

6.1.1. Описание задачи из предметной области

В задачах обработки спутниковых изображений умножение матриц используется для:

- Спектрального анализа многоканальных изображений (преобразование между цветовыми пространствами).
- Метода главных компонент (РСА) для сжатия гиперспектральных данных.
- Линейных преобразований координат при геометрической коррекции снимков.
- Реализации нейросетевых моделей для классификации земного покрова.

Рассмотрим задачу спектрального преобразования спутникового изображения Sentinel-2. Изображение представлено матрицей I размером $H \times W \times C$, где H — высота, W — ширина, C — количество спектральных каналов (13 для Sentinel-2). Требуется применить матрицу преобразования M размером $C' \times C$ для получения нового набора спектральных каналов C' . Операция выполняется как тензорное умножение:

$$I'_{hwc'} = \sum_{c=1}^C I_{hwc} \cdot M_{c'c}. \quad (7)$$

Для изображения Sentinel-2 размером $10980 \times 10980 \times 13$ и $C' = 6$ это составляет порядка 10^{10} операций умножения-сложения.

6.1.2. План решения задачи

1. Реализовать скалярный алгоритм умножения матриц на Python с использованием вложенных циклов (с оптимизацией порядка циклов).
2. Реализовать векторизованную версию с использованием NumPy.
3. Реализовать JIT-компилированную версию с использованием Numba.
4. Реализовать скалярный алгоритм на C++ с использованием вложенных циклов.
5. Реализовать версию с авто-векторизацией компилятора (флаги `-O3 -march=native`).
6. Реализовать версию с использованием интринсиков AVX2.
7. Реализовать блочную версию с оптимизацией кэша.
8. Провести замеры производительности для всех версий.
9. Проанализировать результаты и сформулировать выводы.

6.1.3. Подробное пошаговое описание решения

Шаг 1. Скалярная реализация на Python. Создаём функцию `matmul_scalar_python`. *Важно:* используем порядок циклов `i -> k -> j` для обеспечения пространственной локальности данных при обращении к матрице `B`.

Шаг 2. Векторизованная реализация на Python с NumPy. Используем функцию `numpy.dot`, которая вызывает оптимизированную библиотеку BLAS. Проверяем корректность результатов сравнением со скалярной версией.

Шаг 3. JIT-компиляция с Numba. Применяем декоратор `@jit(parallel=True)` к скалярной функции и заменяем внешний цикл на `prange` для параллельного выполнения.

Шаг 4. Скалярная реализация на C++. Создаём функцию `matmul_scalar` с использованием тройного вложенного цикла. Для хранения матриц используем одномерные массивы с индексацией `i * N + j` для лучшей локальности данных. Порядок циклов: `i -> k -> j`.

Шаг 5. Авто-векторизация C++. Компилируем ту же функцию с флагами `-O3 -march=native -ftree-vectorize`. Добавляем ключевое слово `__restrict` к указателям для устранения псевдонимов.

Шаг 6. Векторизация с интринсиками AVX2. Реализуем функцию `matmul_avx2`. Обработываем по 8 элементов за одну итерацию с использованием FMA. *Обязательно:* добавляем скалярную обработку «хвоста» массива, если размер `N` не кратен 8.

Шаг 7. Блочная версия. Реализуем блочный алгоритм с размером блока 64×64 , помещающимся в кэш L1.

Шаг 8. Замеры производительности. Используем `time.perf_counter()` в Python и `std::chrono::high_resolution_clock` в C++. Для каждого размера матрицы выполняем 10 замеров и вычисляем среднее значение.

Шаг 9. Профилирование. Запускаем программу под `perf stat` для сбора статистики о промахов кэша и утилизации векторных регистров.

6.1.4. Программный код на Python

Фрагмент кода `<code_fragment_1>` содержит скалярную реализацию умножения матриц на Python, фрагмент `<code_fragment_2>` — векторизованную реализацию с NumPy, фрагмент `<code_fragment_3>` — реализацию с Numba.

6.1.5. Программный код на C++

Фрагмент кода `<code_fragment_4>` содержит скалярную реализацию умножения матриц на C++, фрагмент `<code_fragment_5>` — реализацию с авто-векторизацией, фрагмент `<code_fragment_6>` — реализацию с интринсиками AVX2, фрагмент `<code_fragment_7>` — блочную реализацию.

6.1.6. Анализ решения и формулирование выводов

По результатам замеров заполняется таблица:

Размер	Py scalar	NumPy	Numba	C++ scalar	C++ auto	C++ AVX2	C++ block
256×256	...	...	...	...	...	...	...
512×512	...	...	...	...	...	...	...
1024×1024	...	...	...	...	...	...	...
2048×2048	...	...	...	...	...	...	...

Таблица 3: Результаты замеров производительности умножения матриц

На основе данных таблицы строятся графики зависимости времени выполнения от размера матрицы и вычисляется ускорение относительно скалярной реализации.

Ожидаемые результаты:

- NumPy обеспечивает ускорение в 100–1000 раз относительно скалярного Python благодаря BLAS.
- Numba обеспечивает ускорение в 50–200 раз относительно скалярного Python.
- C++ с авто-векторизацией обеспечивает ускорение в 4–8 раз относительно скалярного C++.
- C++ с AVX2 обеспечивает ускорение в 8–12 раз относительно скалярного C++.
- Блочная версия обеспечивает дополнительное ускорение в 1.5–2 раза для больших матриц.

6.2. Задача 2. Двумерная свёртка

6.2.1. Описание задачи из предметной области

Двумерная свёртка — ключевая операция обработки спутниковых изображений. Она применяется для:

- Сглаживания шумов на снимках (фильтр Гаусса).
- Выделения границ и контуров объектов (оператор Собеля, оператор Превитта).
- Обнаружения объектов заданной формы (согласованная фильтрация).
- Улучшения контраста и резкости изображений.
- Морфологических операций (эрозия, дилатация).

Рассмотрим задачу выделения границ на спутниковом изображении сельскохозяйственного поля. Изображение представлено матрицей яркости I размером $H \times W$. Требуется применить оператор Собеля для вычисления градиента яркости:

$$G_x = I * K_x, \quad G_y = I * K_y, \quad |\nabla I| = \sqrt{G_x^2 + G_y^2}. \quad (8)$$

Для изображения размером 10000×10000 пикселей с ядром 3×3 это составляет порядка 10^9 операций.

6.2.2. План решения задачи

1. Реализовать скалярный алгоритм свёртки на Python.
2. Реализовать векторизованную версию с SciPy.
3. Реализовать JIT-компилированную версию с Numba.
4. Реализовать скалярный алгоритм на C++.
5. Реализовать версию с авто-векторизацией.
6. Реализовать версию с интринсиками AVX2 (с обработкой границ и хвоста).
7. Реализовать версию с разделяемым ядром.
8. Провести замеры производительности.
9. Проанализировать результаты.

6.2.3. Подробное пошаговое описание решения

Шаг 1. Скалярная реализация на Python. Реализуем функцию `convolve2d_scalar`, использующую четыре вложенных цикла: два по изображению и два по ядру.

Шаг 2. Векторизованная реализация с SciPy. Используем `scipy.signal.convolve2d(image, kernel, mode='same')`. Проверяем корректность сравнением со скалярной версией.

Шаг 3. JIT-компиляция с Numba. Применяем декоратор `@jit(parallel=True)` к скалярной функции.

Шаг 4. Скалярная реализация на C++. Реализуем функцию `convolve2d_scalar` с использованием четырёх вложенных циклов. Для граничных пикселей используем обработку с ограничением индексов.

Шаг 5. Авто-векторизация C++. Компилируем с флагами `-O3 -march=native`. Проверяем отчёт векторизации.

Шаг 6. Векторизация с интринсиками AVX2. Реализуем функцию `convolve2d_avx2`. Векторизуем внутренний цикл по ширине изображения (j). Разделяем обработку на три части: левая граница (скалярно), середина (векторно), правая граница и хвост (скалярно).

Шаг 7. Разделяемое ядро. Реализуем два прохода: горизонтальный и вертикальный. Для оператора Собеля это даёт ускорение за счёт сокращения числа операций.

Шаг 8. Замеры производительности. Проводим замеры для изображений размером 512×512 , 1024×1024 , 2048×2048 , 4096×4096 .

Шаг 9. Профилирование. Запускаем под `perf stat` для анализа промахов кэша.

6.2.4. Программный код на Python

Фрагмент кода `<code_fragment_8>` содержит скалярную реализацию свёртки на Python, фрагмент `<code_fragment_9>` — реализацию с SciPy, фрагмент `<code_fragment_10>` — реализацию с Numba.

6.2.5. Программный код на C++

Фрагмент кода `<code_fragment_11>` содержит скалярную реализацию свёртки на C++, фрагмент `<code_fragment_12>` — реализацию с авто-векторизацией, фрагмент `<code_fragment_13>` — реализацию с интринсиками AVX2, фрагмент `<code_fragment_14>` — реализацию с разделяемым ядром.

6.2.6. Анализ решения и формулирование выводов

Заполняется таблица результатов:

Размер	Py scalar	SciPy	Numba	C++ scalar	C++ auto	C++ AVX2	C++ separ.
512×512	...	...	...	...	...	...	...
1024×1024	...	...	...	...	...	...	...
2048×2048	...	...	...	...	...	...	...
4096×4096	...	...	...	...	...	...	...

Таблица 4: Результаты замеров производительности двумерной свёртки

Ожидаемые результаты:

- SciPy обеспечивает ускорение в 100–500 раз относительно скалярного Python.
- Numba обеспечивает ускорение в 50–150 раз.
- C++ с авто-векторизацией — в 3–6 раз.
- C++ с AVX2 — в 6–10 раз.
- Разделяемое ядро — в 1.5–2 раза дополнительно.

6.3. Задача 3. Профилирование и оптимизация выравнивания данных

6.3.1. Описание задачи из предметной области

При обработке спутниковых изображений критически важным является эффективное использование кэш-памяти. Изображения высокого разрешения не помещаются в кэш, что приводит к частым промахам и снижению производительности. Выравнивание данных в памяти позволяет использовать векторные инструкции загрузки/сохранения `_mm256_load_ps` вместо `_mm256_loadu_ps`, что даёт ускорение до 20%.

6.3.2. План решения задачи

1. Реализовать блочную обработку изображения с размером блока, помещающимся в кэш.
2. Реализовать выравнивание данных с использованием `aligned_alloc` (C++) и `numpy.zeros` с параметром `order='C'` (Python).

3. Провести профилирование с использованием `perf stat` и `Cachegrind`.
4. Измерить утилизацию векторных регистров с использованием `perf stat -e fp_arith_inst`.
5. Проанализировать влияние выравнивания на производительность.

6.3.3. Подробное пошаговое описание решения

Шаг 1. Блочная обработка. Разбиваем изображение на блоки размером 256×256 , помещающиеся в кэш L2. Для каждого блока применяем свёртку с перекрытием границ.

Шаг 2. Выравнивание данных. В C++ используем `aligned_alloc(32, size)` для выделения памяти, выровненной по 32 байта (граница AVX2). В Python используем `numpy.zeros((H, W), dtype=np.float32, order='C')` для непрерывного размещения.

Шаг 3. Профилирование. Запускаем программу под `perf stat -e cache-references,cache-misses`.

Шаг 4. Анализ утилизации регистров. Сравниваем количество выполненных 256-битных операций с теоретическим максимумом.

Шаг 5. Анализ влияния выравнивания. Сравниваем производительность версий с выровненными и невыровненными данными.

6.3.4. Программный код на Python

Фрагмент кода `<code_fragment_15>` содержит реализацию блочной обработки на Python.

6.3.5. Программный код на C++

Фрагмент кода `<code_fragment_16>` содержит реализацию блочной обработки на C++ с выравниванием данных, фрагмент `<code_fragment_17>` — реализацию с невыровненными данными для сравнения.

6.3.6. Анализ решения и формулирование выводов

Заполняется таблица:

Версия	Время	Cache-misses	Утилизация AVX2
Без выравнивания	...	...	...
С выравниванием	...	...	...
Блочная	...	...	...
Блочная + выравнивание	...	...	...

Таблица 5: Влияние выравнивания и блочной обработки

Ожидаемые результаты:

- Выравнивание данных даёт ускорение 5–15%.
- Блочная обработка снижает количество промахов кэша в 2–5 раз.
- Комбинация методов даёт ускорение 1.2–1.5 раза относительно базовой векторизации.

7. Индивидуальное задание

Студент выбирает вариант задания в соответствии с номером в журнале. Для каждого варианта необходимо:

1. Реализовать указанный алгоритм на Python и C++ в скалярном и векторизованном вариантах.
2. Провести замеры производительности.
3. Построить графики зависимости времени от размера данных.
4. Сформулировать выводы.

Вариант	Алгоритм	Параметры
1	Умножение матриц	$512 \times 512 \times 512$
2	Умножение матриц	$1024 \times 1024 \times 1024$
3	Свёртка	Ядро Гаусса 5×5
4	Свёртка	Оператор Собеля 3×3
5	Свёртка	Оператор Превитта 3×3
6	Свёртка	Ядро усреднения 7×7
7	Умножение матриц	$2048 \times 2048 \times 2048$
8	Свёртка с разделяемым ядром	Ядро Гаусса 9×9
9	Умножение матриц	$256 \times 256 \times 256$
10	Свёртка	Оператор Лапласа 3×3
11	Умножение матриц	$4096 \times 4096 \times 4096$
12	Свёртка	Ядро резкости 5×5
13	Свёртка	Оператор Щарра 3×3
14	Умножение матриц	$128 \times 128 \times 128$
15	Свёртка	Ядро Гаусса 11×11
16	Свёртка	Оператор Робертса 2×2
17	Умножение матриц	$512 \times 1024 \times 512$
18	Свёртка	Ядро медианы 3×3
19	Умножение матриц	$1024 \times 512 \times 1024$
20	Свёртка	Оператор Кирша 3×3
21	Умножение матриц	$2048 \times 1024 \times 2048$
22	Свёртка	Ядро Гаусса 7×7
23	Свёртка	Оператор Собеля с разделением
24	Умножение матриц	$8192 \times 8192 \times 8192$
25	Свёртка	Ядро усреднения 9×9
26	Умножение матриц	$64 \times 64 \times 64$
27	Свёртка	Оператор Лапласа 5×5
28	Свёртка	Ядро резкости 7×7
29	Умножение матриц	$512 \times 512 \times 1024$
30	Свёртка	Оператор Собеля 5×5

Таблица 6: Варианты индивидуальных заданий

8. Тестовые задания с ответами

8.1. Задания с выбором одного варианта из нескольких

Вопрос 1. Какая ширина векторного регистра соответствует набору инструкций AVX2?

- а) 64 бита
- б) 128 бит
- в) 256 бит
- г) 512 бит

Ответ: в) 256 бит.

Вопрос 2. Какая функция NumPy используется для умножения матриц?

- а) `numpy.multiply`
- б) `numpy.dot`
- в) `numpy.matmul`
- г) `numpy.prod`

Ответ: б) `numpy.dot` (также допустимо `numpy.matmul`).

Вопрос 3. Какой флаг GCC включает авто-векторизацию?

- а) `-O1`
- б) `-O2`
- в) `-O3`
- г) `-Os`

Ответ: в) `-O3`.

Вопрос 4. Какая инструкция AVX2 выполняет операцию FMA?

- а) `_mm256_add_ps`
- б) `_mm256_mul_ps`
- в) `_mm256_fmadd_ps`
- г) `_mm256_sub_ps`

Ответ: в) `_mm256_fmadd_ps`.

Вопрос 5. Какой инструмент используется для анализа промахов кэша?

- а) `gdb`
- б) `valgrind -tool=cachegrind`
- в) `strace`
- г) `ltrace`

Ответ: б) `valgrind -tool=cachegrind`.

8.2. Задания с множественным выбором

Вопрос 6. Какие из перечисленных наборов SIMD-инструкций поддерживаются процессорами x86-64?

- а) SSE
- б) AVX
- в) NEON
- г) AVX-512

Ответ: а, б, г.

Вопрос 7. Какие условия необходимы для авто-векторизации цикла?

- а) Отсутствие зависимостей по данным
- б) Использование указателей
- в) Известное количество итераций
- г) Простые типы данных

Ответ: а, в, г.

Вопрос 8. Какие из перечисленных библиотек Python используют SIMD-инструкции?

- а) NumPy
- б) SciPy
- в) Numba
- г) Matplotlib

Ответ: а, б, в.

Вопрос 9. Какие метрики собирает `perf stat`?

- а) `cache-references`
- б) `cache-misses`
- в) `instructions`
- г) `memory-usage`

Ответ: а, б, в.

Вопрос 10. Какие факторы влияют на производительность умножения матриц?

- а) Размер матрицы
- б) Выравнивание данных
- в) Цвет элементов матрицы
- г) Размер кэша
- д) Порядок вложенных циклов (Loop Interchange)

Ответ: а, б, г, д.

8.3. Задания на сопоставление

Вопрос 11. Установите соответствие между набором инструкций и шириной регистра:

- 1) SSE
- 2) AVX
- 3) AVX2
- 4) AVX-512
 - а) 512 бит
- б) 128 бит
- в) 256 бит
- г) 256 бит (расширенный)

Ответ: 1 – б, 2 – в, 3 – г, 4 – а.

Вопрос 12. Установите соответствие между функцией NumPy и её назначением:

- 1) `numpy.dot`
- 2) `numpy.convolve`
- 3) `numpy.einsum`
- 4) `numpy.transpose`
 - а) Транспонирование
- б) Умножение матриц
- в) Выражения Эйнштейна
- г) Свёртка

Ответ: 1 – б, 2 – г, 3 – в, 4 – а.

Вопрос 13. Установите соответствие между интринсиком и его назначением:

- 1) `_mm256_load_ps`
- 2) `_mm256_store_ps`
- 3) `_mm256_set1_ps`
- 4) `_mm256_fmadd_ps`
 - а) Заполнение вектора
- б) Загрузка данных

в) FMA

г) Сохранение данных

Ответ: 1 – б, 2 – г, 3 – а, 4 – в.

Вопрос 14. Установите соответствие между инструментом и его назначением:

1) `perf stat`

2) `perf record`

3) `Cachegrind`

4) `gprof`

а) Запись профиля

б) Статистика производительности

в) Анализ кэша

г) Профилирование вызовов

Ответ: 1 – б, 2 – а, 3 – в, 4 – г.

Вопрос 15. Установите соответствие между оператором выделения границ и его ядром:

1) Собель

2) Превитт

3) Робертс

4) Лаплас

а) 2×2

б) 3×3 (сглаживающий)

в) 3×3 (лапласиан)

г) 3×3 (стандартный)

Ответ: 1 – г, 2 – б, 3 – а, 4 – в.

8.4. Задания на ввод пропущенного слова

Вопрос 16. Набор инструкций, обеспечивающий 512-битные векторные операции, называется _____.

Ответ: AVX-512.

Вопрос 17. Для загрузки выровненных данных в AVX2 используется интринсик `_mm256_`_____.

Ответ: `load`.

Вопрос 18. Операция, объединяющая умножение и сложение в одну инструкцию, называется _____.

Ответ: FMA (Fused Multiply-Add).

Вопрос 19. Инструмент Valgrind для анализа промахов кэша называется _____.

Ответ: Cachegrind.

Вопрос 20. Функция NumPy для вычисления выражений Эйнштейна называется _____.

Ответ: `einsum`.

8.5. Задания на анализ фрагмента программного кода

Вопрос 21. Проанализируйте фрагмент кода `<code_fragment_18>`. Какое ускорение обеспечивает данный фрагмент относительно скалярной реализации?

Ответ: Фрагмент реализует умножение матриц с использованием AVX2. Обработывается по 8 элементов за итерацию, что даёт теоретическое ускорение до 8 раз. Также исправлен порядок циклов (i-k-j), что устраняет промахи кэша и даёт реальное ускорение по сравнению с наивным порядком (i-j-k).

Вопрос 22. Проанализируйте фрагмент кода `<code_fragment_19>`. Какая ошибка допущена при загрузке данных?

Ответ: Используется `_mm256_load_ps` для потенциально невыровненных данных. Необходимо использовать `_mm256_loadu_ps` или обеспечить выравнивание.

Вопрос 23. Проанализируйте фрагмент кода `<code_fragment_20>`. Какие оптимизации применены?

Ответ: Применены блочная обработка, порядок циклов i-k-j для улучшения локальности, выравнивание данных.

Вопрос 24. Проанализируйте фрагмент кода `<code_fragment_21>`. Какое ускорение обеспечивает Numba относительно скалярного Python?

Ответ: Numba обеспечивает ускорение в 50–200 раз благодаря JIT-компиляции и параллельному выполнению.

Вопрос 25. Проанализируйте фрагмент кода `<code_fragment_22>`. Какая ошибка допущена при обработке граничных пикселей?

Ответ: Отсутствует проверка выхода за границы изображения, что приводит к чтению некорректных данных (Segmentation Fault).

8.6. Задания на поиск ошибок в фрагменте программного кода

Вопрос 26. Найдите ошибку в фрагменте кода `<code_fragment_23>`.

Ответ: Отсутствует освобождение памяти, выделенной через `aligned_alloc`. Необходимо вызвать `free(ptr)`.

Вопрос 27. Найдите ошибку в фрагменте кода `<code_fragment_24>`.

Ответ: Неверный порядок индексов при доступе к матрице: `B[j * N + k]` вместо `B[k * N + j]` при условии, что матрица `B` хранится по строкам. Это приводит к нелокальному доступу к памяти.

Вопрос 28. Найдите ошибку в фрагменте кода `<code_fragment_25>`.

Ответ: Использование `prange` без параметра `parallel=True` в декораторе `@njit`.

Вопрос 29. Найдите ошибку в фрагменте кода `<code_fragment_26>`.

Ответ: Неверное выравнивание памяти: `aligned_alloc(16, size)` вместо `aligned_alloc(32, size)` для AVX2.

Вопрос 30. Найдите ошибку в фрагменте кода `<code_fragment_27>`.

Ответ: Отсутствует обработка остатка при векторизации: если размер массива не кратен 8, последние элементы не обрабатываются, что приводит к потере данных или `segfault`.

9. Контрольные вопросы

1. Что такое SIMD и какие преимущества он предоставляет при обработке изображений?
2. Перечислите основные наборы SIMD-инструкций процессоров x86-64 и укажите их различия.
3. Какие условия необходимы для успешной авто-векторизации цикла компилятором?
4. В чём различие между интринсиками `_mm256_load_ps` и `_mm256_loadu_ps`?
5. Как операция FMA повышает производительность вычислений?
6. Опишите математическую модель двумерной свёртки и её вычислительную сложность. В чём отличие свёртки от кросс-корреляции?

7. В чём преимущество разделяемых ядер при вычислении двумерной свёртки?
8. Какие метрики производительности собирает `perf stat` и как их интерпретировать?
9. Как выравнивание данных в памяти влияет на производительность векторизованных алгоритмов?
10. Что такое модель Roofline? Приведите примеры алгоритмов, ограниченных вычислениями (Compute-bound) и памятью (Memory-bound).
11. Почему порядок вложенных циклов (Loop Interchange) критически важен для производительности умножения матриц в C/C++?
12. Какие рекомендации можно сформулировать по оптимизации алгоритмов обработки спутниковых изображений?

А. Фрагменты программного кода

А.1. Фрагмент 1. Скалярное умножение матриц на Python

```
1  import numpy as np
2  import time
3
4  def matmul_scalar_python(A, B):
5      """Scalar matrix multiplication in Python with loop order optimization."""
6
7      M, K = A.shape
8      K2, N = B.shape
9      assert K == K2, "Incompatible matrix dimensions"
10
11      C = np.zeros((M, N), dtype=np.float32)
12      # Loop order i -> k -> j for spatial locality
13      for i in range(M):
14          for k in range(K):
15              a = A[i, k]
16              for j in range(N):
17                  C[i, j] += a * B[k, j]
18      return C
19
20      np.random.seed(42)
21      N = 256
22      A = np.random.rand(N, N).astype(np.float32)
23      B = np.random.rand(N, N).astype(np.float32)
24
25      start = time.perf_counter()
26      C = matmul_scalar_python(A, B)
27      elapsed = time.perf_counter() - start
28      print(f"Scalar Python: {elapsed:.4f} s")
```

А.2. Фрагмент 2. Векторизованное умножение матриц с NumPy

```
1  import numpy as np
2  import time
3
4  np.random.seed(42)
5  N = 256
```

```

6     A = np.random.rand(N, N).astype(np.float32)
7     B = np.random.rand(N, N).astype(np.float32)
8
9     start = time.perf_counter()
10    C = np.dot(A, B)
11    elapsed = time.perf_counter() - start
12    print(f"NumPy: {elapsed:.6f} s")

```

А.3. Фрагмент 3. JIT-компиляция умножения матриц с Numba

```

1     import numpy as np
2     from numba import njit, prange
3     import time
4
5     @njit(parallel=True)
6     def matmul_numba(A, B):
7         """JIT-compiled matrix multiplication."""
8         M, K = A.shape
9         N = B.shape[1]
10        C = np.zeros((M, N), dtype=np.float32)
11        for i in prange(M):
12            for k in range(K):
13                a = A[i, k]
14                for j in range(N):
15                    C[i, j] += a * B[k, j]
16        return C
17
18    np.random.seed(42)
19    N = 256
20    A = np.random.rand(N, N).astype(np.float32)
21    B = np.random.rand(N, N).astype(np.float32)
22
23    matmul_numba(A, B) # Warm-up
24
25    start = time.perf_counter()
26    C = matmul_numba(A, B)
27    elapsed = time.perf_counter() - start
28    print(f"Numba: {elapsed:.6f} s")

```

А.4. Фрагмент 4. Скалярное умножение матриц на C++

```

1     #include <iostream>
2     #include <vector>
3     #include <chrono>
4     #include <random>
5     #include <algorithm>
6
7     void matmul_scalar(const float* A, const float* B, float* C, int M, int
8         K, int N) {
9         for (int i = 0; i < M; ++i) {
10            for (int k = 0; k < K; ++k) {
11                float a = A[i * K + k];
12                for (int j = 0; j < N; ++j) {
13                    C[i * N + j] += a * B[k * N + j];
14                }
15            }
16        }
17    }

```

```

16     }
17
18     int main() {
19         int M = 256, K = 256, N = 256;
20         std::vector<float> A(M * K), B(K * N), C(M * N, 0.0f);
21         std::mt19937 gen(42);
22         std::uniform_real_distribution<float> dist(0.0f, 1.0f);
23         for (auto& x : A) x = dist(gen);
24         for (auto& x : B) x = dist(gen);
25
26         auto start = std::chrono::high_resolution_clock::now();
27         matmul_scalar(A.data(), B.data(), C.data(), M, K, N);
28         auto end = std::chrono::high_resolution_clock::now();
29         std::chrono::duration<double> elapsed = end - start;
30         std::cout << "C++ scalar: " << elapsed.count() << " s\n";
31         return 0;
32     }

```

A.5. Фрагмент 5. Умножение матриц с авто-векторизацией C++

```

1 // Compilation: g++ -O3 -march=native -ftree-vectorize \
2 // -fopt-info-vec-optimized matmul.cpp -o matmul
3
4 #include <iostream>
5 #include <vector>
6 #include <chrono>
7 #include <random>
8
9 void matmul_auto(const float* __restrict A, const float* __restrict B,
10 float* __restrict C, int M, int K, int N) {
11     for (int i = 0; i < M; ++i) {
12         for (int k = 0; k < K; ++k) {
13             float a = A[i * K + k];
14             #pragma omp simd
15             for (int j = 0; j < N; ++j) {
16                 C[i * N + j] += a * B[k * N + j];
17             }
18         }
19     }
20 }
21
22 int main() {
23     int M = 256, K = 256, N = 256;
24     std::vector<float> A(M * K), B(K * N), C(M * N, 0.0f);
25     std::mt19937 gen(42);
26     std::uniform_real_distribution<float> dist(0.0f, 1.0f);
27     for (auto& x : A) x = dist(gen);
28     for (auto& x : B) x = dist(gen);
29
30     auto start = std::chrono::high_resolution_clock::now();
31     matmul_auto(A.data(), B.data(), C.data(), M, K, N);
32     auto end = std::chrono::high_resolution_clock::now();
33     std::chrono::duration<double> elapsed = end - start;
34     std::cout << "C++ auto: " << elapsed.count() << " s\n";
35     return 0;
36 }

```

А.6. Фрагмент 6. Умножение матриц с интринсиками AVX2

```

1  #include <immintrin.h>
2  #include <iostream>
3  #include <vector>
4  #include <chrono>
5  #include <random>
6
7  void matmul_avx2(const float* __restrict A, const float* __restrict B,
8  float* __restrict C, int M, int K, int N) {
9      for (int i = 0; i < M; ++i) {
10         for (int k = 0; k < K; ++k) {
11             __m256 a = _mm256_set1_ps(A[i * K + k]);
12             int j = 0;
13             // Vectorized part
14             for (; j <= N - 8; j += 8) {
15                 __m256 b = _mm256_loadu_ps(&B[k * N + j]);
16                 __m256 c = _mm256_loadu_ps(&C[i * N + j]);
17                 __m256 res = _mm256_fmadd_ps(a, b, c);
18                 _mm256_storeu_ps(&C[i * N + j], res);
19             }
20             // Tail processing
21             for (; j < N; ++j) {
22                 C[i * N + j] += A[i * K + k] * B[k * N + j];
23             }
24         }
25     }
26 }
27
28 int main() {
29     int M = 256, K = 256, N = 256;
30     std::vector<float> A(M * K), B(K * N), C(M * N, 0.0f);
31     std::mt19937 gen(42);
32     std::uniform_real_distribution<float> dist(0.0f, 1.0f);
33     for (auto& x : A) x = dist(gen);
34     for (auto& x : B) x = dist(gen);
35
36     auto start = std::chrono::high_resolution_clock::now();
37     matmul_avx2(A.data(), B.data(), C.data(), M, K, N);
38     auto end = std::chrono::high_resolution_clock::now();
39     std::chrono::duration<double> elapsed = end - start;
40     std::cout << "C++ AVX2: " << elapsed.count() << " s\n";
41     return 0;
42 }

```

А.7. Фрагмент 7. Блочное умножение матриц на C++

```

1  #include <iostream>
2  #include <vector>
3  #include <chrono>
4  #include <random>
5  #include <cstring>
6  #include <algorithm>
7
8  constexpr int BLOCK = 64;
9
10 void matmul_blocked(const float* A, const float* B, float* C, int M, int
11 K, int N) {
    std::memset(C, 0, M * N * sizeof(float));

```

```

12     for (int ii = 0; ii < M; ii += BLOCK) {
13         for (int kk = 0; kk < K; kk += BLOCK) {
14             for (int jj = 0; jj < N; jj += BLOCK) {
15                 for (int i = ii; i < std::min(ii + BLOCK, M); ++i) {
16                     for (int k = kk; k < std::min(kk + BLOCK, K); ++k) {
17                         float a = A[i * K + k];
18                         for (int j = jj; j < std::min(jj + BLOCK, N); ++
19                             j) {
20                             C[i * N + j] += a * B[k * N + j];
21                         }
22                     }
23                 }
24             }
25         }
26     }
27
28     int main() {
29         int M = 256, K = 256, N = 256;
30         std::vector<float> A(M * K), B(K * N), C(M * N);
31         std::mt19937 gen(42);
32         std::uniform_real_distribution<float> dist(0.0f, 1.0f);
33         for (auto& x : A) x = dist(gen);
34         for (auto& x : B) x = dist(gen);
35
36         auto start = std::chrono::high_resolution_clock::now();
37         matmul_blocked(A.data(), B.data(), C.data(), M, K, N);
38         auto end = std::chrono::high_resolution_clock::now();
39         std::chrono::duration<double> elapsed = end - start;
40         std::cout << "C++ blocked: " << elapsed.count() << " s\n";
41         return 0;
42     }

```

A.8. Фрагмент 8. Скалярная свёртка на Python

```

1     import numpy as np
2     import time
3
4     def convolve2d_scalar(image, kernel):
5         """Scalar 2D convolution."""
6         H, W = image.shape
7         kh, kw = kernel.shape
8         ph, pw = kh // 2, kw // 2
9         result = np.zeros_like(image, dtype=np.float32)
10
11         for i in range(H):
12             for j in range(W):
13                 s = 0.0
14                 for ki in range(kh):
15                     for kj in range(kw):
16                         ii = i + ki - ph
17                         jj = j + kj - pw
18                         if 0 <= ii < H and 0 <= jj < W:
19                             s += image[ii, jj] * kernel[ki, kj]
20                 result[i, j] = s
21         return result
22
23     np.random.seed(42)
24     image = np.random.rand(512, 512).astype(np.float32)

```

```

25     kernel = np.array([[1, 2, 1], [2, 4, 2], [1, 2, 1]], dtype=np.float32) /
        16.0
26
27     start = time.perf_counter()
28     result = convolve2d_scalar(image, kernel)
29     elapsed = time.perf_counter() - start
30     print(f"Scalar Python: {elapsed:.4f} s")

```

A.9. Фрагмент 9. Свёртка с SciPy

```

1     import numpy as np
2     from scipy.signal import convolve2d
3     import time
4
5     np.random.seed(42)
6     image = np.random.rand(512, 512).astype(np.float32)
7     kernel = np.array([[1, 2, 1], [2, 4, 2], [1, 2, 1]], dtype=np.float32) /
        16.0
8
9     start = time.perf_counter()
10    result = convolve2d(image, kernel, mode='same')
11    elapsed = time.perf_counter() - start
12    print(f"SciPy: {elapsed:.6f} s")

```

A.10. Фрагмент 10. Свёртка с Numba

```

1     import numpy as np
2     from numba import njit, prange
3     import time
4
5     @njit(parallel=True)
6     def convolve2d_numba(image, kernel):
7         H, W = image.shape
8         kh, kw = kernel.shape
9         ph, pw = kh // 2, kw // 2
10        result = np.zeros_like(image)
11        for i in prange(H):
12            for j in range(W):
13                s = 0.0
14                for ki in range(kh):
15                    for kj in range(kw):
16                        ii = i + ki - ph
17                        jj = j + kj - pw
18                        if 0 <= ii < H and 0 <= jj < W:
19                            s += image[ii, jj] * kernel[ki, kj]
20                result[i, j] = s
21        return result
22
23    np.random.seed(42)
24    image = np.random.rand(512, 512).astype(np.float32)
25    kernel = np.array([[1, 2, 1], [2, 4, 2], [1, 2, 1]], dtype=np.float32) /
        16.0
26
27    convolve2d_numba(image, kernel)
28
29    start = time.perf_counter()
30    result = convolve2d_numba(image, kernel)

```

```

31     elapsed = time.perf_counter() - start
32     print(f"Numba: {elapsed:.6f} s")

```

A.11. Фрагмент 11. Скалярная свёртка на C++

```

1     #include <iostream>
2     #include <vector>
3     #include <chrono>
4     #include <random>
5     #include <cmath>
6
7     void convolve2d_scalar(const float* image, const float* kernel, float*
      result,
8     int H, int W, int kh, int kw) {
9         int ph = kh / 2, pw = kw / 2;
10        for (int i = 0; i < H; ++i) {
11            for (int j = 0; j < W; ++j) {
12                float s = 0.0f;
13                for (int ki = 0; ki < kh; ++ki) {
14                    for (int kj = 0; kj < kw; ++kj) {
15                        int ii = i + ki - ph;
16                        int jj = j + kj - pw;
17                        if (ii >= 0 && ii < H && jj >= 0 && jj < W) {
18                            s += image[ii * W + jj] * kernel[ki * kw + kj];
19                        }
20                    }
21                }
22                result[i * W + j] = s;
23            }
24        }
25    }
26
27    int main() {
28        int H = 512, W = 512, kh = 3, kw = 3;
29        std::vector<float> image(H * W), kernel(kh * kw), result(H * W);
30        std::mt19937 gen(42);
31        std::uniform_real_distribution<float> dist(0.0f, 1.0f);
32        for (auto& x : image) x = dist(gen);
33        float k[9] = {1, 2, 1, 2, 4, 2, 1, 2, 1};
34        for (int i = 0; i < 9; ++i) kernel[i] = k[i] / 16.0f;
35
36        auto start = std::chrono::high_resolution_clock::now();
37        convolve2d_scalar(image.data(), kernel.data(), result.data(), H, W,
38        kh, kw);
39        auto end = std::chrono::high_resolution_clock::now();
40        std::chrono::duration<double> elapsed = end - start;
41        std::cout << "C++ scalar: " << elapsed.count() << " s\n";
42        return 0;
43    }

```

A.12. Фрагмент 12. Свёртка с авто-векторизацией C++

```

1     // Compilation: g++ -O3 -march=native -ftree-vectorize \
2     // -fopt-info-vec-optimized conv.cpp
3
4     #include <iostream>
5     #include <vector>

```

```

6      #include <chrono>
7      #include <random>
8
9      void convolve2d_auto(const float* __restrict image, const float*
10         __restrict kernel,
11         float* __restrict result, int H, int W, int kh, int kw) {
12         int ph = kh / 2, pw = kw / 2;
13         for (int i = 0; i < H; ++i) {
14             for (int j = 0; j < W; ++j) {
15                 float s = 0.0f;
16                 for (int ki = 0; ki < kh; ++ki) {
17                     for (int kj = 0; kj < kw; ++kj) {
18                         int ii = i + ki - ph;
19                         int jj = j + kj - pw;
20                         if (ii >= 0 && ii < H && jj >= 0 && jj < W) {
21                             s += image[ii * W + jj] * kernel[ki * kw + kj];
22                         }
23                     }
24                 }
25                 result[i * W + j] = s;
26             }
27         }
28
29         int main() {
30             int H = 512, W = 512, kh = 3, kw = 3;
31             std::vector<float> image(H * W), kernel(kh * kw), result(H * W);
32             std::mt19937 gen(42);
33             std::uniform_real_distribution<float> dist(0.0f, 1.0f);
34             for (auto& x : image) x = dist(gen);
35             float k[9] = {1, 2, 1, 2, 4, 2, 1, 2, 1};
36             for (int i = 0; i < 9; ++i) kernel[i] = k[i] / 16.0f;
37
38             auto start = std::chrono::high_resolution_clock::now();
39             convolve2d_auto(image.data(), kernel.data(), result.data(), H, W, kh
40                 , kw);
41             auto end = std::chrono::high_resolution_clock::now();
42             std::chrono::duration<double> elapsed = end - start;
43             std::cout << "C++ auto: " << elapsed.count() << " s\n";
44             return 0;
45         }

```

А.13. Фрагмент 13. Свёртка с интринсиками AVX2

```

1      #include <immintrin.h>
2      #include <iostream>
3      #include <vector>
4      #include <chrono>
5      #include <random>
6      #include <algorithm>
7
8      void convolve2d_avx2(const float* image, const float* kernel, float*
9         result,
10         int H, int W, int kh, int kw) {
11         int ph = kh / 2, pw = kw / 2;
12         for (int i = 0; i < H; ++i) {
13             int j_start = std::max(0, pw);
14             int j_end = std::min(W, W - pw);

```

```

15 // Left border (scalar)
16 for (int j = 0; j < j_start; ++j) {
17     float s = 0.0f;
18     for (int ki = 0; ki < kh; ++ki) {
19         for (int kj = 0; kj < kw; ++kj) {
20             int ii = i + ki - ph;
21             int jj = j + kj - pw;
22             if (ii >= 0 && ii < H && jj >= 0 && jj < W) {
23                 s += image[ii * W + jj] * kernel[ki * kw + kj];
24             }
25         }
26     }
27     result[i * W + j] = s;
28 }
29
30 // Vectorized middle
31 int j = j_start;
32 for (; j <= j_end - 8; j += 8) {
33     __m256 sum = _mm256_setzero_ps();
34     for (int ki = 0; ki < kh; ++ki) {
35         int ii = i + ki - ph;
36         if (ii < 0 || ii >= H) continue;
37         for (int kj = 0; kj < kw; ++kj) {
38             __m256 kvec = _mm256_set1_ps(kernel[ki * kw + kj]);
39             __m256 img_vec = _mm256_loadu_ps(&image[ii * W + j +
40                 kj - pw]);
41             sum = _mm256_fmadd_ps(kvec, img_vec, sum);
42         }
43     }
44     _mm256_storeu_ps(&result[i * W + j], sum);
45 }
46
47 // Right border and tail (scalar)
48 for (; j < W; ++j) {
49     float s = 0.0f;
50     for (int ki = 0; ki < kh; ++ki) {
51         for (int kj = 0; kj < kw; ++kj) {
52             int ii = i + ki - ph;
53             int jj = j + kj - pw;
54             if (ii >= 0 && ii < H && jj >= 0 && jj < W) {
55                 s += image[ii * W + jj] * kernel[ki * kw + kj];
56             }
57         }
58     }
59     result[i * W + j] = s;
60 }
61 }
62
63 int main() {
64     int H = 512, W = 512, kh = 3, kw = 3;
65     std::vector<float> image(H * W), kernel(kh * kw), result(H * W);
66     std::mt19937 gen(42);
67     std::uniform_real_distribution<float> dist(0.0f, 1.0f);
68     for (auto& x : image) x = dist(gen);
69     float k[9] = {1, 2, 1, 2, 4, 2, 1, 2, 1};
70     for (int i = 0; i < 9; ++i) kernel[i] = k[i] / 16.0f;
71
72     auto start = std::chrono::high_resolution_clock::now();

```

```

73     convolve2d_avx2(image.data(), kernel.data(), result.data(), H, W, kh
74         , kw);
75     auto end = std::chrono::high_resolution_clock::now();
76     std::chrono::duration<double> elapsed = end - start;
77     std::cout << "C++ AVX2: " << elapsed.count() << " s\n";
78     return 0;

```

A.14. Фрагмент 14. Свёртка с разделяемым ядром на C++

```

1     #include <iostream>
2     #include <vector>
3     #include <chrono>
4     #include <random>
5
6     void convolve1d_horizontal(const float* src, float* dst, int H, int W,
7         const float* kernel, int kw) {
8         int pw = kw / 2;
9         for (int i = 0; i < H; ++i) {
10             for (int j = 0; j < W; ++j) {
11                 float s = 0.0f;
12                 for (int kj = 0; kj < kw; ++kj) {
13                     int jj = j + kj - pw;
14                     if (jj >= 0 && jj < W) {
15                         s += src[i * W + jj] * kernel[kj];
16                     }
17                 }
18                 dst[i * W + j] = s;
19             }
20         }
21     }
22
23     void convolve1d_vertical(const float* src, float* dst, int H, int W,
24         const float* kernel, int kh) {
25         int ph = kh / 2;
26         for (int i = 0; i < H; ++i) {
27             for (int j = 0; j < W; ++j) {
28                 float s = 0.0f;
29                 for (int ki = 0; ki < kh; ++ki) {
30                     int ii = i + ki - ph;
31                     if (ii >= 0 && ii < H) {
32                         s += src[ii * W + j] * kernel[ki];
33                     }
34                 }
35                 dst[i * W + j] = s;
36             }
37         }
38     }
39
40     void convolve2d_separable(const float* image, float* result, int H, int
41         W,
42         const float* kx, const float* ky, int kh, int kw) {
43         std::vector<float> temp(H * W);
44         convolve1d_horizontal(image, temp.data(), H, W, kx, kw);
45         convolve1d_vertical(temp.data(), result, H, W, ky, kh);
46     }
47
48     int main() {
49         int H = 512, W = 512, kh = 3, kw = 3;

```

```

49     std::vector<float> image(H * W), result(H * W);
50     float kx[3] = {1.0f/4, 2.0f/4, 1.0f/4};
51     float ky[3] = {1.0f/4, 2.0f/4, 1.0f/4};
52     std::mt19937 gen(42);
53     std::uniform_real_distribution<float> dist(0.0f, 1.0f);
54     for (auto& x : image) x = dist(gen);
55
56     auto start = std::chrono::high_resolution_clock::now();
57     convolve2d_separable(image.data(), result.data(), H, W, kx, ky, kh,
58                          kw);
59     auto end = std::chrono::high_resolution_clock::now();
60     std::chrono::duration<double> elapsed = end - start;
61     std::cout << "C++ separable: " << elapsed.count() << " s\n";
62     return 0;
}

```

A.15. Фрагмент 15. Блочная обработка на Python

```

1  import numpy as np
2  from scipy.signal import convolve2d
3
4  def convolve2d_blocked(image, kernel, block_size=256):
5      """Blocked image processing to minimize cache misses."""
6      H, W = image.shape
7      kh, kw = kernel.shape
8      ph, pw = kh // 2, kw // 2
9      result = np.zeros_like(image, dtype=np.float32)
10
11     for i0 in range(0, H, block_size):
12         for j0 in range(0, W, block_size):
13             i1 = min(i0 + block_size, H)
14             j1 = min(j0 + block_size, W)
15
16             # Extend block by kernel borders
17             ie0 = max(0, i0 - ph)
18             ie1 = min(H, i1 + ph)
19             je0 = max(0, j0 - pw)
20             je1 = min(W, j1 + pw)
21
22             block = image[ie0:ie1, je0:je1]
23             conv = convolve2d(block, kernel, mode='same')
24
25             # Extract central part
26             offset_i = i0 - ie0
27             offset_j = j0 - je0
28             result[i0:i1, j0:j1] = conv[offset_i:offset_i + (i1 - i0),
29                                         offset_j:offset_j + (j1 - j0)]
30     return result

```

A.16. Фрагмент 16. Блочная обработка с выравниванием на C++

```

1  #include <immintrin.h>
2  #include <cstdlib>
3  #include <cstring>
4  #include <iostream>
5  #include <chrono>
6  #include <random>

```

```

7
8 void convolve2d_blocked_aligned(const float* image, const float* kernel,
9 float* result, int H, int W, int kh, int kw,
10 int block_size) {
11     int ph = kh / 2, pw = kw / 2;
12     for (int i0 = 0; i0 < H; i0 += block_size) {
13         for (int j0 = 0; j0 < W; j0 += block_size) {
14             int i1 = std::min(i0 + block_size, H);
15             int j1 = std::min(j0 + block_size, W);
16             for (int i = i0; i < i1; ++i) {
17                 for (int j = j0; j < j1; ++j) {
18                     float s = 0.0f;
19                     for (int ki = 0; ki < kh; ++ki) {
20                         for (int kj = 0; kj < kw; ++kj) {
21                             int ii = i + ki - ph;
22                             int jj = j + kj - pw;
23                             if (ii >= 0 && ii < H && jj >= 0 && jj < W)
24                                 s += image[ii * W + jj] * kernel[ki * kw
25                                     + kj];
26                         }
27                     }
28                     result[i * W + j] = s;
29                 }
30             }
31         }
32     }
33 }
34
35 int main() {
36     int H = 512, W = 512, kh = 3, kw = 3;
37     size_t bytes = H * W * sizeof(float);
38     float* image = static_cast<float*>(std::aligned_alloc(32, bytes));
39     float* result = static_cast<float*>(std::aligned_alloc(32, bytes));
40     float kernel[9] = {1.0f/16, 2.0f/16, 1.0f/16, 2.0f/16, 4.0f/16, 2.0f
41         /16,
42         1.0f/16, 2.0f/16, 1.0f/16};
43     std::mt19937 gen(42);
44     std::uniform_real_distribution<float> dist(0.0f, 1.0f);
45     for (int i = 0; i < H * W; ++i) image[i] = dist(gen);
46
47     auto start = std::chrono::high_resolution_clock::now();
48     convolve2d_blocked_aligned(image, kernel, result, H, W, kh, kw, 256)
49     ;
50     auto end = std::chrono::high_resolution_clock::now();
51     std::chrono::duration<double> elapsed = end - start;
52     std::cout << "C++ blocked+aligned: " << elapsed.count() << " s\n";
53
54     std::free(image);
55     std::free(result);
56     return 0;
57 }

```

A.17. Фрагмент 17. Блочная обработка без выравнивания на C++

```

1 #include <iostream>
2 #include <vector>
3 #include <chrono>

```

```

4      #include <random>
5
6      void convolve2d_blocked_unaligned(const float* image, const float*
7      kernel,
8      float* result, int H, int W, int kh, int kw,
9      int block_size) {
10         int ph = kh / 2, pw = kw / 2;
11         for (int i0 = 0; i0 < H; i0 += block_size) {
12             for (int j0 = 0; j0 < W; j0 += block_size) {
13                 int i1 = std::min(i0 + block_size, H);
14                 int j1 = std::min(j0 + block_size, W);
15                 for (int i = i0; i < i1; ++i) {
16                     for (int j = j0; j < j1; ++j) {
17                         float s = 0.0f;
18                         for (int ki = 0; ki < kh; ++ki) {
19                             for (int kj = 0; kj < kw; ++kj) {
20                                 int ii = i + ki - ph;
21                                 int jj = j + kj - pw;
22                                 if (ii >= 0 && ii < H && jj >= 0 && jj < W)
23                                     {
24                                         s += image[ii * W + jj] * kernel[ki * kw
25                                         + kj];
26                                     }
27                                 }
28                             }
29                         }
30                     }
31                 }
32             }
33             result[i * W + j] = s;
34         }
35     }
36
37     int main() {
38         int H = 512, W = 512, kh = 3, kw = 3;
39         std::vector<float> image(H * W), result(H * W);
40         float kernel[9] = {1.0f/16, 2.0f/16, 1.0f/16, 2.0f/16, 4.0f/16, 2.0f
41         /16,
42         1.0f/16, 2.0f/16, 1.0f/16};
43         std::mt19937 gen(42);
44         std::uniform_real_distribution<float> dist(0.0f, 1.0f);
45         for (auto& x : image) x = dist(gen);
46
47         auto start = std::chrono::high_resolution_clock::now();
48         convolve2d_blocked_unaligned(image.data(), kernel, result.data(),
49         H, W, kh, kw, 256);
50         auto end = std::chrono::high_resolution_clock::now();
51         std::chrono::duration<double> elapsed = end - start;
52         std::cout << "C++ blocked+unaligned: " << elapsed.count() << " s\n";
53         return 0;
54     }

```

A.18. Фрагмент 18. Фрагмент для анализа (задание 21)

```

1      void matmul_avx2_analysis(const float* A, const float* B, float* C, int
2      M, int K, int N) {
3          for (int i = 0; i < M; ++i) {
4              for (int k = 0; k < K; ++k) { // Optimized loop order
5                  _mm256 a = _mm256_set1_ps(A[i * K + k]);
6                  int j = 0;

```

```

6         for (; j <= N - 8; j += 8) {
7             __m256 b = _mm256_loadu_ps(&B[k * N + j]);
8             __m256 c = _mm256_loadu_ps(&C[i * N + j]);
9             __m256 res = _mm256_fmadd_ps(a, b, c);
10            __m256_storeu_ps(&C[i * N + j], res);
11        }
12        for (; j < N; ++j) {
13            C[i * N + j] += A[i * K + k] * B[k * N + j];
14        }
15    }
16 }
17

```

A.19. Фрагмент 19. Фрагмент для анализа (задание 22)

```

1 void convolve_buggy(const float* image, float* result, int H, int W) {
2     for (int i = 0; i < H; ++i) {
3         for (int j = 0; j <= W - 8; j += 8) {
4             __m256 vec = _mm256_load_ps(&image[i * W + j]); // Error
5             __m256_store_ps(&result[i * W + j], vec);
6         }
7     }
8 }

```

A.20. Фрагмент 20. Фрагмент для анализа (задание 23)

```

1 void matmul_optimized(const float* A, const float* B, float* C, int M,
2 int K, int N) {
3     constexpr int BLOCK = 64;
4     std::memset(C, 0, M * N * sizeof(float));
5     for (int ii = 0; ii < M; ii += BLOCK) {
6         for (int kk = 0; kk < K; kk += BLOCK) {
7             for (int jj = 0; jj < N; jj += BLOCK) {
8                 for (int i = ii; i < std::min(ii + BLOCK, M); ++i) {
9                     for (int k = kk; k < std::min(kk + BLOCK, K); ++k) {
10                        float a = A[i * K + k];
11                        for (int j = jj; j < std::min(jj + BLOCK, N); ++
12                            j) {
13                            C[i * N + j] += a * B[k * N + j];
14                        }
15                    }
16                }
17            }
18        }
19    }
20 }

```

A.21. Фрагмент 21. Фрагмент для анализа (задание 24)

```

1 @njit(parallel=True)
2 def convolve_numba(image, kernel):
3     H, W = image.shape
4     result = np.zeros_like(image)
5     for i in prange(H):
6         for j in range(W):

```

```
7     result[i, j] = image[i, j]
8     return result
```

A.22. Фрагмент 22. Фрагмент для анализа (задание 25)

```
1     void convolve_no_bounds(const float* image, const float* kernel, float*
2         result,
3         int H, int W, int kh, int kw) {
4         int ph = kh / 2, pw = kw / 2;
5         for (int i = 0; i < H; ++i) {
6             for (int j = 0; j < W; ++j) {
7                 float s = 0.0f;
8                 for (int ki = 0; ki < kh; ++ki) {
9                     for (int kj = 0; kj < kw; ++kj) {
10                         s += image[(i + ki - ph) * W + (j + kj - pw)] *
11                             kernel[ki * kw + kj];
12                             // Error: missing bounds check
13                     }
14                 }
15                 result[i * W + j] = s;
16             }
17         }
```

A.23. Фрагмент 23. Фрагмент с ошибкой (задание 26)

```
1     int main() {
2         size_t bytes = 1024 * 1024 * sizeof(float);
3         float* ptr = static_cast<float*>(std::aligned_alloc(32, bytes));
4         // ... using ptr
5         // Error: missing free(ptr)
6         return 0;
7     }
```

A.24. Фрагмент 24. Фрагмент с ошибкой (задание 27)

```
1     void matmul_wrong_index(const float* A, const float* B, float* C, int M,
2         int K, int N) {
3         for (int i = 0; i < M; ++i) {
4             for (int j = 0; j < N; ++j) {
5                 float s = 0.0f;
6                 for (int k = 0; k < K; ++k) {
7                     s += A[i * K + k] * B[j * N + k]; // Error: wrong index
8                     order
9                 }
10                 C[i * N + j] = s;
11             }
12         }
```

A.25. Фрагмент 25. Фрагмент с ошибкой (задание 28)

```

1  from numba import njit, prange
2
3  @njit # Error: missing parallel=True
4  def matmul_wrong(A, B):
5      M, K = A.shape
6      N = B.shape[1]
7      C = np.zeros((M, N))
8      for i in prange(M):
9          for j in range(N):
10             s = 0.0
11             for k in range(K):
12                 s += A[i, k] * B[k, j]
13             C[i, j] = s
14         return C

```

A.26. Фрагмент 26. Фрагмент с ошибкой (задание 29)

```

1  int main() {
2      size_t bytes = 1024 * 1024 * sizeof(float);
3      // Error: 16-byte alignment instead of 32 for AVX2
4      float* ptr = static_cast<float*>(std::aligned_alloc(16, bytes));
5      // ... usage with _mm256_load_ps
6      std::free(ptr);
7      return 0;
8  }

```

A.27. Фрагмент 27. Фрагмент с ошибкой (задание 30)

```

1  void matmul_no_tail(const float* A, const float* B, float* C, int M, int
   K, int N) {
2      for (int i = 0; i < M; ++i) {
3          for (int j = 0; j < N; j += 8) {
4              __m256 sum = _mm256_setzero_ps();
5              for (int k = 0; k < K; ++k) {
6                  __m256 a = _mm256_set1_ps(A[i * K + k]);
7                  __m256 b = _mm256_loadu_ps(&B[k * N + j]);
8                  sum = _mm256_fmadd_ps(a, b, sum);
9              }
10             __m256_storeu_ps(&C[i * N + j], sum);
11             // Error: missing tail processing if N is not a multiple of
               8
12         }
13     }
14 }

```